A3-PS012-TC292

Authorised DNS Resolver Exposure Assessment

Rwitabrato Hanpal, Anushka Ghosh

rwitabratohanpal1@gmail.com

Team Binary Beans, IEM Kolkata

Mentor: Prof (Dr) Rupayan Das

AIORI Hackathon 3

October 11, 2026

Problem Description

Open recursive DNS resolvers can be abused as reflectors in DDoS amplification attacks, and operators rarely know which of their servers are exposed.

- Open resolvers answer recursive queries from anyone. An attacker sends small queries with a spoofed source address, and the much larger replies hit the victim [2][3].
- Existing checks are ad hoc: a single *dig* from one machine, or reading the recursion-available flag, which forwarders and misconfigured servers can get wrong.
- Testing must itself be safe: no internet-wide scanning, no spoofed traffic and no amplification or stress testing.
- **Goal:** a bounded, reproducible tool that probes up to 20 authorised lab DNS servers from a separate client, classifies each as Recursive, Authoritative-only, Refused or Inconclusive, and compares the result with seeded ground truth.

Background

A recursive resolver looks names up for its clients; an authoritative server answers only for the zones it hosts.

- **Header fields that matter:** RD (recursion desired), RA (recursion available), AA (authoritative answer) and RCODE (NOERROR, REFUSED, SERVFAIL).
- **Reflection:** the attacker forges the victim's address as the query source, so the resolver sends its reply to the victim. A small query can trigger a much larger response [2][3].
- **Best practice:** a resolver should serve only its intended clients. RFC 5358 (BCP 140) recommends restricting recursion by access control [1].
- **Scope:** recursion exposure is the precondition for this abuse, and it is the only property this project measures.
- **Why flags are not enough:** a forwarder or a fake resolver can set the RA flag without really resolving, so we need proof that recursion happened.

Literature & RFC Review

Only the most relevant standards and studies:

- **RFC 5358 (BCP 140)** [1]: restrict recursion to the intended clients. Our main reference and the source of the remediation advice in the report.
- **RFC 1035, 6891, 7766** [5][6][7]: message format, EDNS payload size and TCP fallback for truncated replies. They define how probes are built and retried.
- **RFC 8906** [8]: common operational DNS failures (timeouts, broken EDNS). Helps explain Inconclusive results.
- **Kührer et al., Rossow** [2][3]: how reflection and amplification abuse open resolvers, and why closing them helps.
- **Park et al., APNIC** [4][9]: open resolvers behave inconsistently (forwarders, deviating headers, altered answers), so flags alone mislead.
- **Gap:** prior work measures the whole internet. We run a controlled, rate-limited assessment of authorised servers, validated against ground truth, with proof of recursion at our own authoritative server.

Proposed Solution

A probe tool that proves recursion instead of inferring it from flags.

1. **Safety gate:** probes only targets listed in a scope file (owner, authoriser, date), maximum 20.
2. **Rate control:** token bucket (global and per target), about 3 queries per target, kill switch.
3. **Prober** (Python, dnspython): a recursive query for a unique random name (nonce) in our test zone, plus a query for the zone the target should serve. UDP first, TCP if the reply is truncated.
4. **Proof of recursion:** our own authoritative server for the test zone logs every name it receives. If the nonce shows up there, the target really recursed.
5. **Classifier: Recursive** (valid answer and nonce seen), **Authoritative-only** (own zone answered with AA, outside name refused or unanswered), **Refused** (REFUSED on both probes), **Inconclusive** (timeout, SERVFAIL, bad reply or conflicting evidence, reason stored).
6. **Comparator and report:** labels vs seeded ground truth (confusion matrix, false and inconclusive analysis); JSON, CSV and HTML output with RFC 5358 remediation.
7. **Lab:** Docker Compose with 20 seeded servers (BIND, Unbound, Knot, dnsmasq, NSD, PowerDNS, a forwarder and fault cases) and a client on a separate network.

Optimization Proposed

Design choices that make the assessment more reliable and safer than a one-off *dig*.

Aspect	Before (naive check)	After (our design)
Detection	Reads the RA flag only	RA flag plus the nonce seen at our authoritative server
Forwarders	Mistaken for real recursion	Matched on the nonce, not on the source address
Network load	Repeated, unbounded queries	Token bucket, per-target query cap, kill switch
Reproducibility	Manual, differs from run to run	Config-driven, seeded lab, logged evidence
Truncated replies	Treated as a failure	Automatic TCP fallback
Unclear results	Forced into a class	Explicit Inconclusive with a stored reason
Validation	None	Ground-truth comparison and error analysis

Planned further enhancements: rate limits verified with a packet capture, and a fix-and-rescan step that shows a Recursive server turning into Refused.

Solution Architecture & Design

Blue: client-side tool • **Orange: safety gates** • **Green: authorised lab and data**

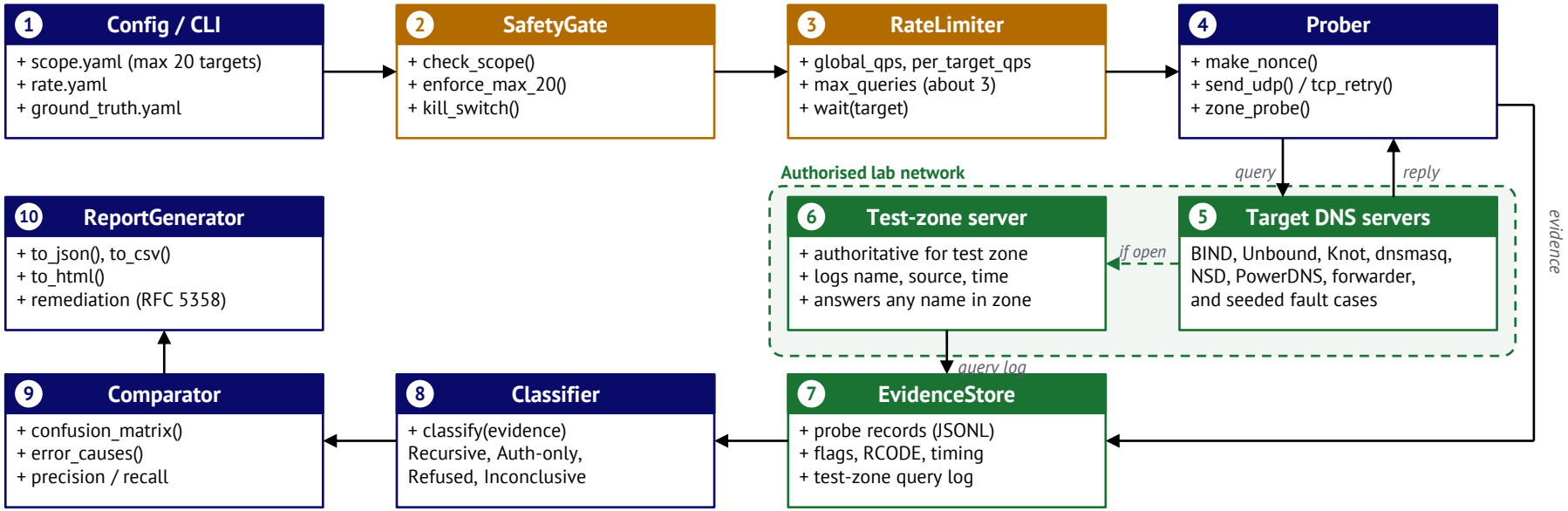


Figure: System Architecture UML Diagram

Data Flow Diagram

Seven processes, four data stores, three external entities. Dashed: lab recursion.

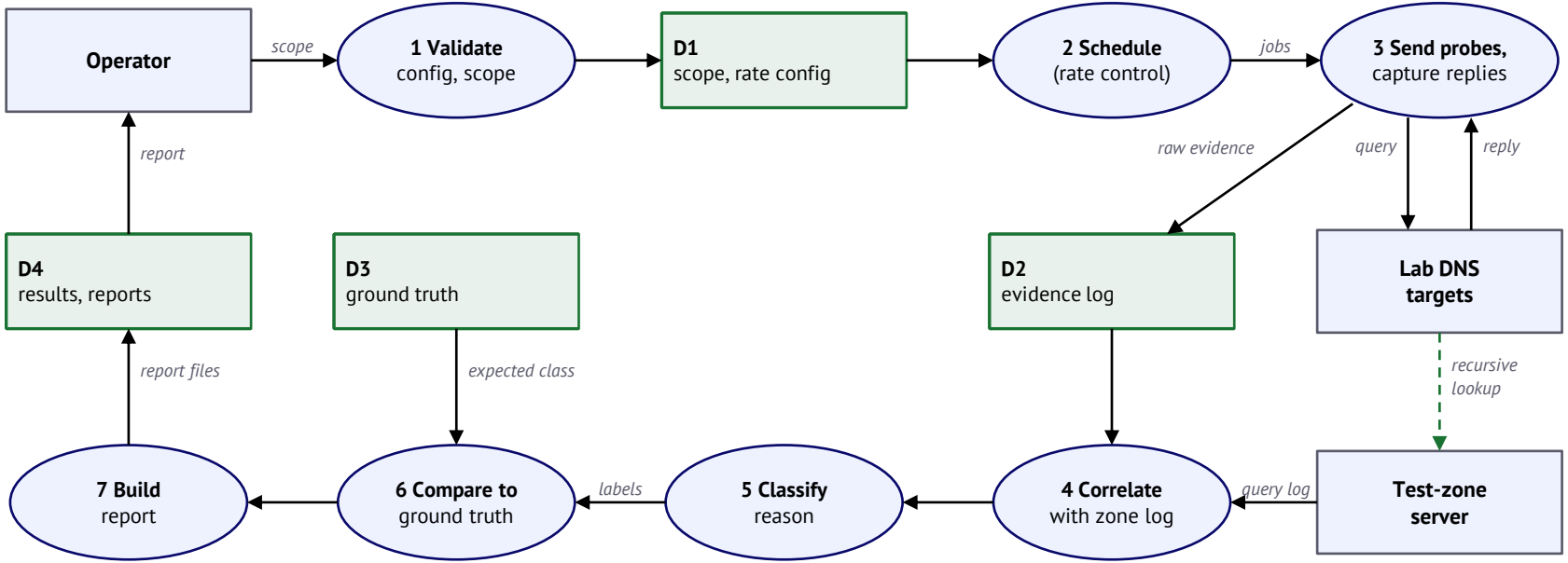


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Project milestones for the online phase, 16–31 October, each with a clear exit test.

Dates	Milestone and deliverable	Done when
16–19 Oct	**Lab and ground truth:** Docker Compose lab, test-zone server, 20 seeded targets	All 20 targets start with one command; a manual probe shows in the test-zone log
20–23 Oct	**Prober and safety:** nonce probe, UDP/TCP retry, scope check, rate limiter, kill switch	A full 20-target run stays inside the rate caps
24–27 Oct	**Classifier and comparator:** four verdicts with reasons, confusion matrix, error analysis	Every miss has a documented root cause
28–29 Oct	**Reports and verification:** JSON, CSV and HTML, packet-capture check of rate limits, 3 repeat runs	Labels agree across the three runs
30–31 Oct	**Demo and submission:** one-command demo, README, pseudocode snippet, code freeze	An evaluator can reproduce the demo from the repository

Methodology: lab first, so every component is built and tested against a known right answer; small increments, with one Git branch per milestone, unit tests for the classifier rules and an end-to-end run against the lab after each; mentor feedback folded in at every milestone review; reproducible by design (pinned container images, recorded run seed and config hash, one-command demo).

Bibliography

[1] J. Damas and F. Neves. “Preventing Use of Recursive Nameservers in Reflector Attacks.” In: *RFC 5358 (BCP 140)*, IETF (2008).

[2] M. Kührer, T. Hupperich, C. Rossow and T. Holz. “Exit from Hell? Reducing the Impact of Amplification DDoS Attacks.” In: *USENIX Security Symposium* (2014).

[3] C. Rossow. “Amplification Hell: Revisiting Network Protocols for DDoS Abuse.” In: *Network and Distributed System Security Symposium (NDSS)* (2014).

[4] J. Park et al. “Where Are You Taking Me? Behavioral Analysis of Open DNS Resolvers.” In: *IEEE/IFIP DSN* (2019), pp. 493–504.

[5] P. Mockapetris. “Domain Names – Implementation and Specification.” In: *RFC 1035*, IETF (1987).

[6] J. Damas, M. Graff and P. Vixie. “Extension Mechanisms for DNS (EDNS(0)).” In: *RFC 6891*, IETF (2013).

[7] J. Dickinson et al. “DNS Transport over TCP – Implementation Requirements.” In: *RFC 7766*, IETF (2016).

[8] M. Andrews and R. Bellis. “A Common Operational Problem in DNS Servers: Failure to Communicate.” In: *RFC 8906 (BCP 231)*, IETF (2020).

[9] APNIC Blog. “Open DNS resolvers, from bad to worse.” 13 May 2022.

Thank You for Your Attention!